FULL-STACK WEB DEVELOPMENT PROJECT

# Todo List Application

Make today a little lighter.

| | |
|---|---|
| **Prepared by** | Shifa Khan |
| **Project type** | Full-stack Todo List web application |
| **Live application** | todo-app-mfnx.onrender.com |
| **Source code** | github.com/Shifa-Khan-15/Todo-App |

A practical project in task management, database integration, authentication, and AI-assisted planning.

# 01 | Project Overview

## Abstract

The Todo List Application is a full-stack web project created to make everyday task planning simpler, clearer, and more organised. Through a personal workspace, users can register for an account and log in, create new tasks, edit task descriptions, mark tasks as complete, delete tasks, and filter the list to focus on all, pending, or completed work. The application also includes an AI Task Breakdown feature, which can turn a larger goal into smaller suggested actions and help users decide where to begin. The interface is built with HTML, CSS, and vanilla JavaScript, while Node.js and Express handle server-side requests and application logic. MongoDB Atlas stores application data, with Mongoose providing the model-based database connection. The project brings these parts together in a deployed web application and demonstrates how interface design, backend processing, database persistence, account access, and an external AI service can work as one system.

## Introduction

People often keep reminders in different notes, messages, or memory, which can make it difficult to see what needs attention and what has already been completed. A digital task list offers one place to record work, review progress, and update plans when priorities change. This project was developed as a practical full-stack web application rather than a static checklist. It combines a responsive browser interface with server routes, account authentication, and a cloud database so that task information can be managed through a connected application. Its visual approach uses a calm, editorial style and the phrase “Make today a little lighter” to keep the experience focused and approachable.

## Project purpose

The purpose of the project is to demonstrate the complete flow of a modern web application: a user performs an action in the frontend, the frontend sends a request to the backend, the backend processes the request and communicates with the database, and the updated result is returned to the interface. The project covers core CRUD operations (Create, Read, Update, Delete), user account access, task-status filtering, cloud data storage, and AI-assisted planning. It also provides practical experience with project structure, environment variables, deployment, and keeping credentials out of public source code. The AI-generated steps are suggestions for planning and remain under the user's control.

# Contents

PROJECT DOCUMENTATION


| | | |
|---|---|---|
| 01 | Project Overview | 2 |
| 02 | Objectives and Problem Statement | 4 |
| 03 | Technology Stack and System Design | 5 |
| 04 | Functional Features | 6 |
| 05 | Screenshots and User Workflow | 7 |
| 06 | Testing and Results | 12 |
| 07 | Setup and Configuration | 13 |
| 08 | Limitations and Future Scope | 14 |
| 09 | Conclusion and References | 15 |

# 02 | Objectives and Problem Statement

## Problem statement

Daily tasks can become difficult to manage when they are scattered across notes, messages, or memory. A basic checklist may also be inconvenient when a user wants to edit tasks, keep them after closing the browser, or break a larger task into manageable actions. This project addresses these needs through a single web application.

## Objectives

- Build a clean, responsive interface that is comfortable to use on mobile and desktop screens.
- Implement account registration, login, and logout flows.
- Support the main task operations: add, view, edit, complete, and delete.
- Provide filters for all tasks, pending tasks, and completed tasks.
- Store application data in a cloud-hosted MongoDB database.
- Integrate AI-assisted task breakdown to suggest smaller steps for a larger goal.
- Practise the connection between frontend code, backend routes, database models, and deployment.

## Project scope

The application focuses on personal task organisation. Its current scope includes authentication, task management, status filtering, cloud persistence, and AI-generated step suggestions. Features such as calendar reminders and due-date scheduling are potential future enhancements.

# 03 | Technology Stack and System Design

## Technology stack

| Layer | Technology | Role in the project |
|---|---|---|
| Frontend | HTML5, CSS3, Vanilla JavaScript | Displays the interface and handles user interactions, task controls, filters, and AI feature inputs. |
| Backend | Node.js, Express.js | Receives requests, applies application logic, handles authentication, and communicates with the database. |
| Database | MongoDB Atlas, Mongoose | Stores application records in a cloud database and provides a model-based way to work with MongoDB. |
| AI service | Gemini API | Generates suggested steps for a user-entered task through the AI Task Breakdown feature. |
| Deployment and source control | Render, GitHub | Render hosts the live application; GitHub stores and tracks the project source code. |

## High-level architecture

| **1. User**<br>Mobile or desktop browser | **2. Frontend**<br>HTML · CSS · JavaScript | **3. Backend**<br>Node.js · Express |
|---|---|---|
| Interacts with the interface and submits actions. | Sends requests and displays responses. | Validates and processes requests, then accesses services. |

The backend communicates with MongoDB Atlas for stored data and with the Gemini API for AI task suggestions. Environment variables hold sensitive configuration values so they do not need to be written directly into source code.

## Project file structure

| File or folder | Purpose |
|---|---|
| `public/index.html` | Main page structure and interface content. |
| `public/style.css` | Visual design, colours, spacing, typography, and responsive layout. |
| `public/app.js` | Task interactions, filters, and AI Task Breakdown interface behaviour. |
| `public/auth.js` | Frontend login and registration interactions. |
| `server.js` | Backend server and application routes. |
| `package.json` / `package-lock.json` | Project metadata and dependency records. |

# 04 | Functional Features

## 1. Login and registration

The authentication screen follows the same visual language as the main app: a warm off-white background, deep green typography, soft borders, and terracotta action buttons. A user can create an account or sign in to reach the personal task workspace.

## 2. Task management

- **Add:** Enter a task in the input field and use the plus button to add it to the list.
- **Complete:** Select the checkbox to mark a task complete. Completed text appears with a line-through treatment.
- **Edit:** Use the Edit control to change the task description.
- **Delete:** Use the Delete control to remove a task.

## 3. Status filters

The All, To do, and Done tabs allow users to view the full list, pending tasks, or completed tasks. This keeps the list easier to scan as the number of tasks grows.

## 4. AI Task Breakdown

The user enters a larger task, such as preparing a science project, and selects “Break Down with AI.” The app displays suggested steps. The “Add All to My Todo List” action allows the suggested steps to be transferred into the task list, reducing the effort required to plan a large activity.

## 5. Cloud persistence and connection status

The application uses MongoDB Atlas for cloud storage. The interface includes a connection indicator and a status message for the task list. Data availability depends on a working server and database connection.

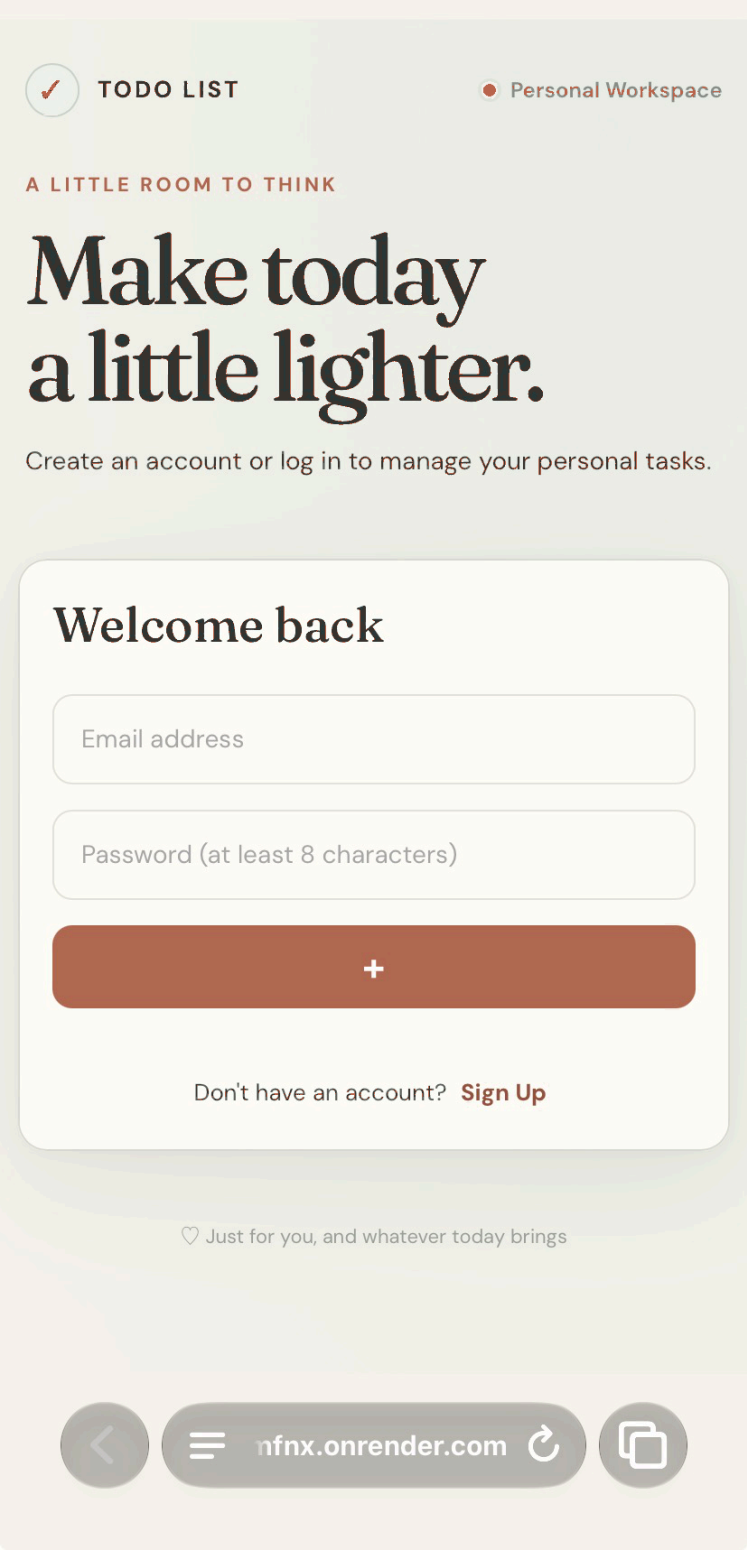
## Design language

The interface uses the phrase “Make today a little lighter” and a calm editorial style. Its main visual choices are deep green headings, muted sage accents, warm cream surfaces, terracotta buttons, rounded cards, and generous spacing. The documentation follows the same palette to make the report feel connected to the product.

# 05 | Screenshots and User Workflow

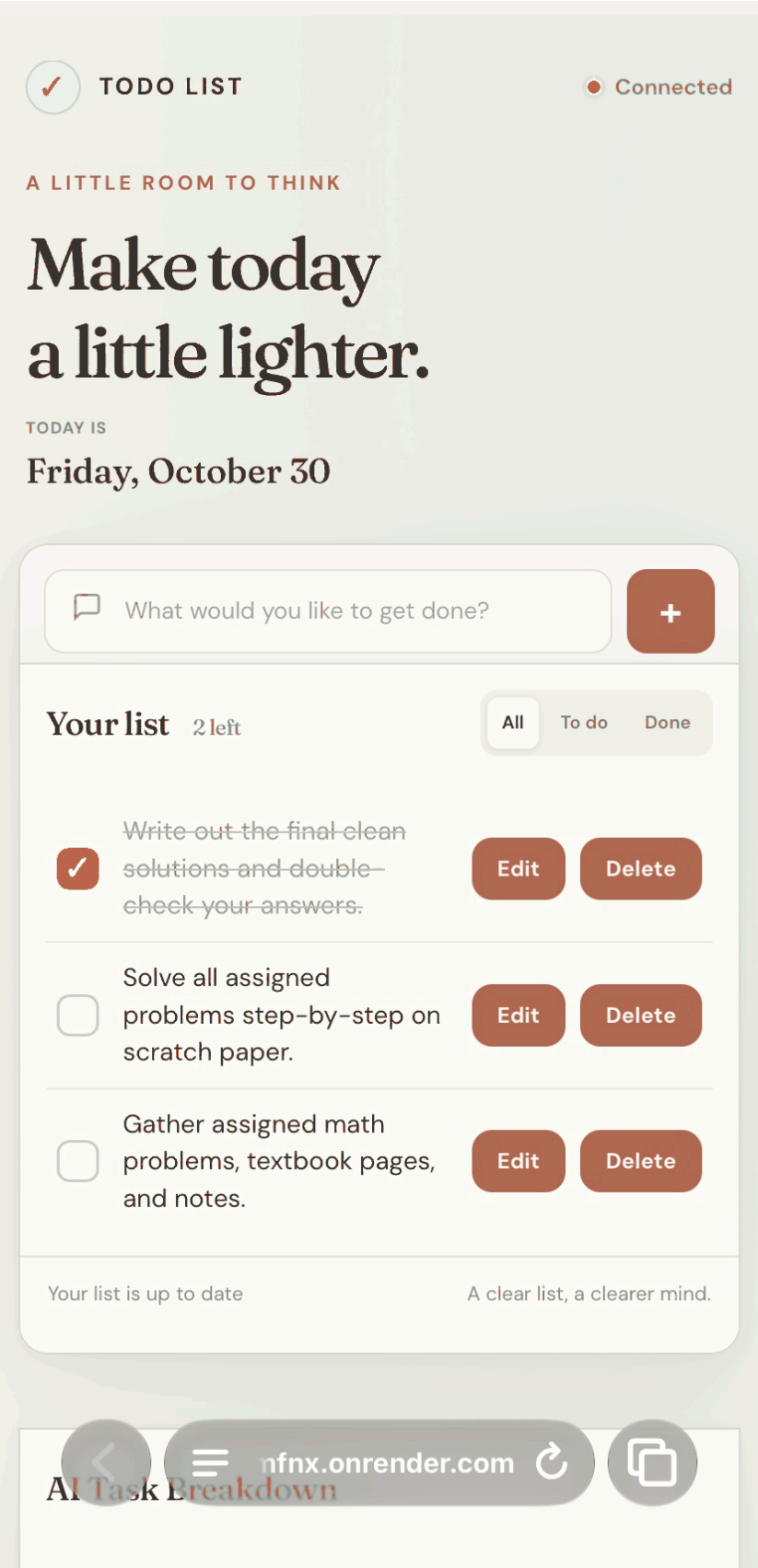
The following screenshots are provided from the live application. They illustrate the main authentication screen, task workspace, task filters, and AI-generated task suggestions.

## Screenshot A — Account access



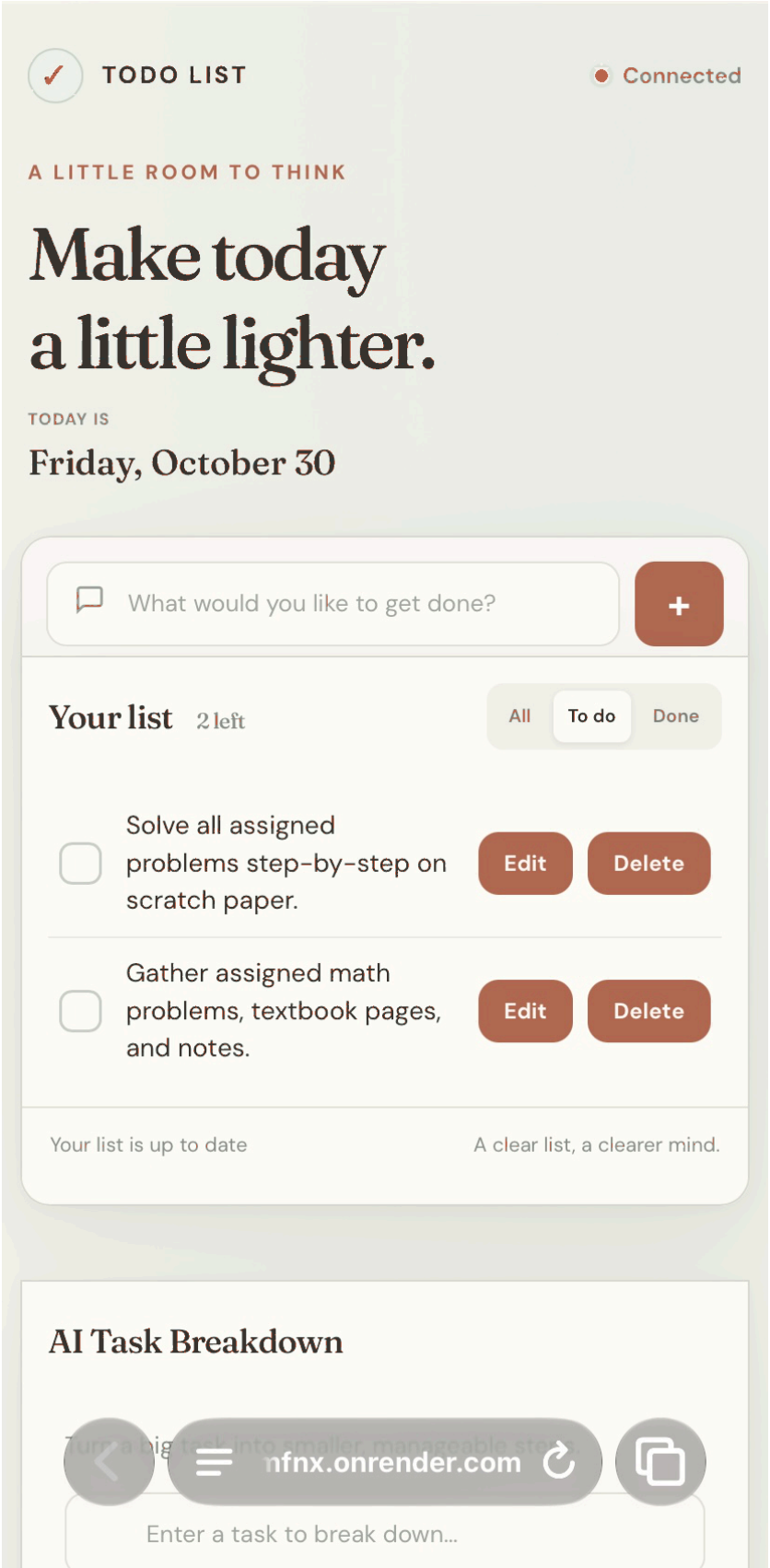

Login screen with account access, personal-workspace label

## Screenshot B — Main task workspace



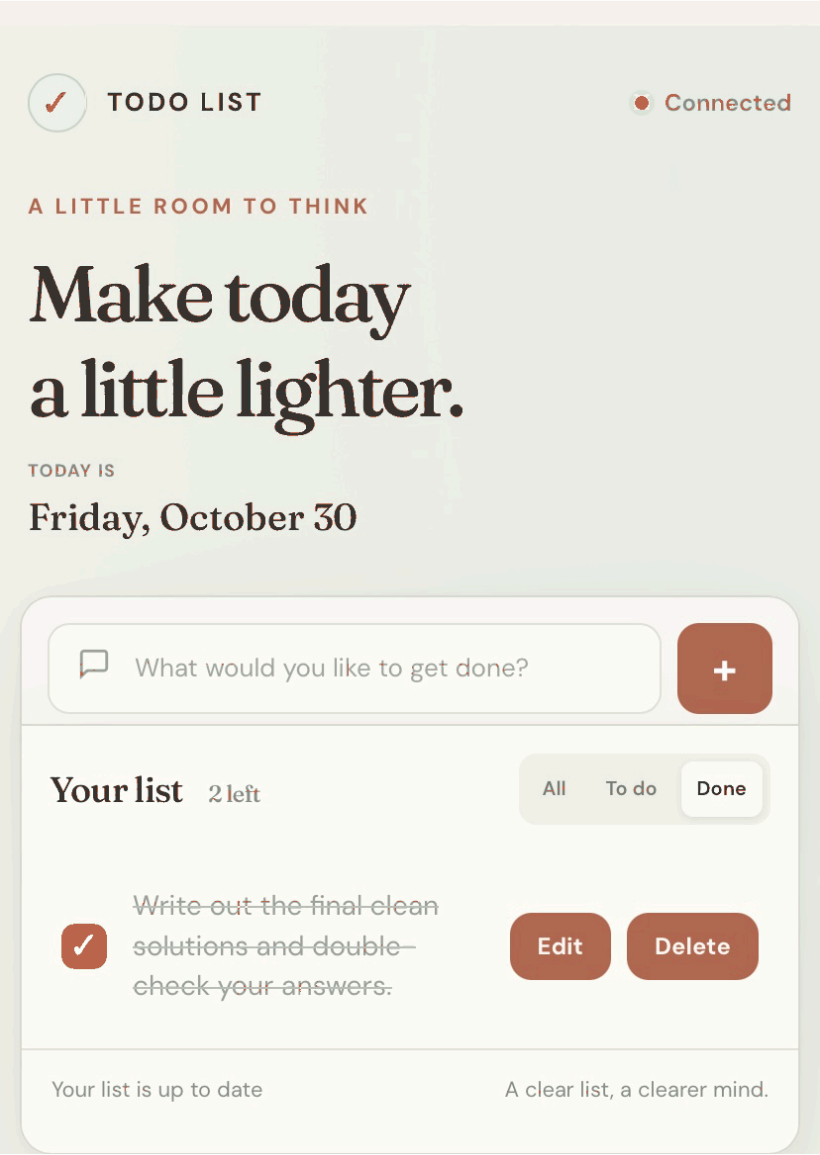

Main task list with task entry, completion controls, Edit/Delete actions, and the All filter.

## Screenshot C — Pending tasks filter



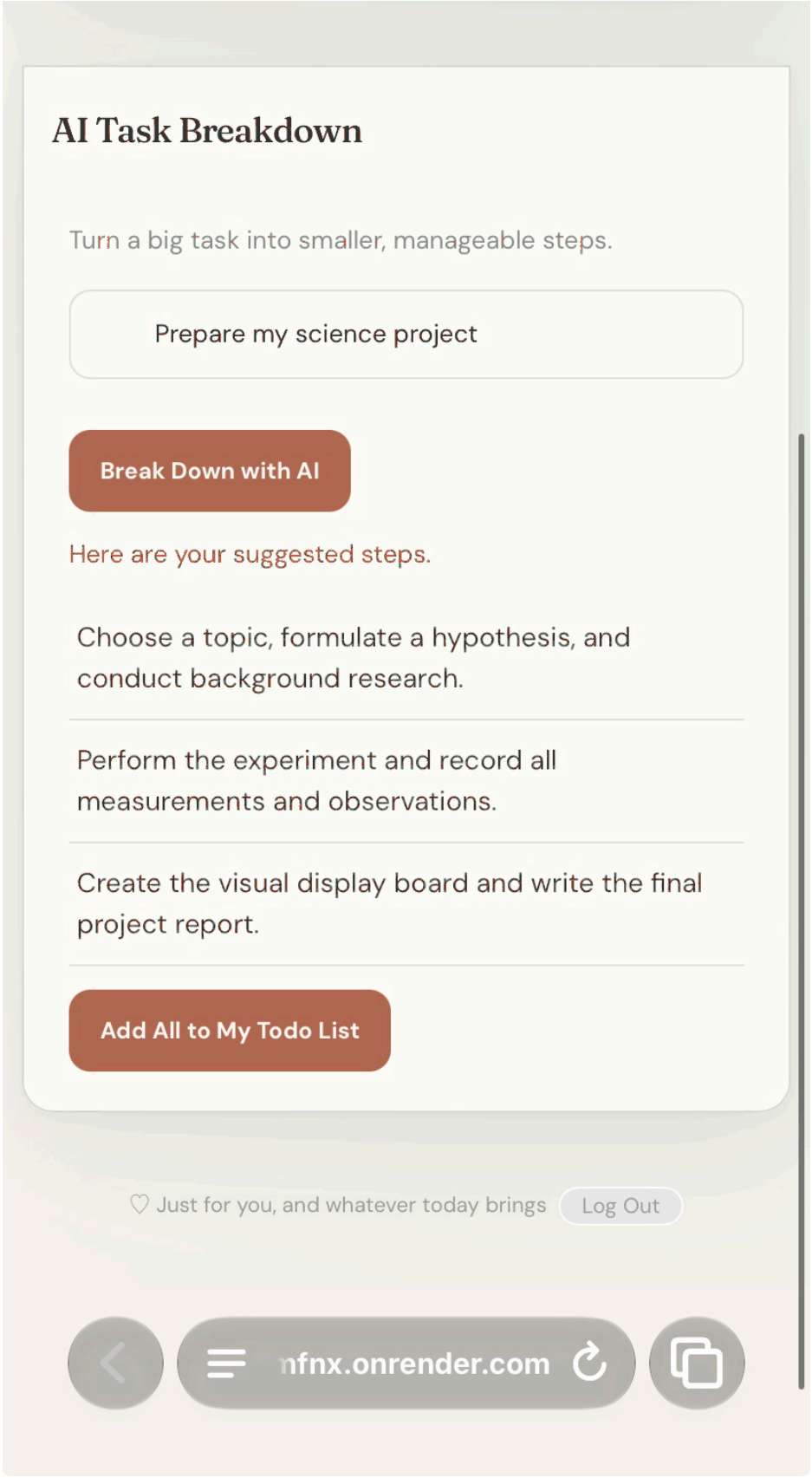

The To do filter shows the tasks that remain incomplete.

## Screenshot D — Completed tasks




The Done filter displays completed work

## Screenshot E — AI Task Breakdown result




Example AI result: a larger task is converted into suggested steps, with an option to add all steps to the Todo List.

# 06 | Testing and Results

## Testing approach

The table below is a practical checklist for validating the app's main user journeys. It describes the expected behaviour; it should be treated as a test plan rather than a claim that every test has been independently run as part of this document.

| Test area | Action | Expected result |
|---|---|---|
| Registration | Submit valid new account details. | Account is created and the user can proceed to the app. |
| Login / logout | Log in, then log out. | Access changes according to the user's session state. |
| Add task | Enter a task and submit it. | The task appears in the list and is stored by the backend. |
| Complete task | Select a task checkbox. | The task's completed state and styling update. |
| Edit task | Change an existing task title. | The updated task text is displayed and saved. |
| Delete task | Delete a task. | The selected task is removed from the list. |
| Filters | Select All, To do, and Done. | The list shows tasks matching the selected status. |
| AI breakdown | Submit a larger task to the AI feature. | Suggested steps are shown; user can add them to the list. |
| Persistence | Refresh or reopen the app after saving a task. | Saved tasks remain available when the server and database are functioning. |

## Observed interface outcome

The supplied screenshots show a responsive, mobile-friendly interface, task status filters, completed-task styling, and a working example of AI-generated steps. The live application is available through Render, and the project source is maintained on GitHub.

## Known operational dependencies

- MongoDB Atlas must be reachable and the connection string must be configured correctly.
- The Gemini API key must be valid for AI Task Breakdown.
- The hosting service and application server must be available for the live URL to respond.

# 07 | Setup and Configuration

## Requirements

- A supported Node.js installation with npm.
- Access to the project repository on GitHub.
- A MongoDB Atlas connection string.
- A Gemini API key for the AI feature.

## Run locally

1. Clone the repository:

```
git clone https://github.com/Shifa-Khan-15/Todo-App.git
```

2. Enter the project directory:

```
cd Todo-App
```

3. Install dependencies:

```
npm install
```

4. Create a local `.env` file in the project root with the required values:

```
MONGO_URI=your_mongodb_connection_string
SESSION_SECRET=your_session_secret
GEMINI_API_KEY=your_gemini_api_key
PORT=3000
```

5. Start the server:

```
node server.js
```

6. Open `http://localhost:3000` in a browser after the server starts successfully.

## Security and configuration notes

- Use your own valid credentials; the example values above are placeholders.
- Do not commit the .env file or publish database passwords and API keys.
- The environment-variable names in this guide must match the names read by the server code and the deployment settings.
- A working database connection is required for stored tasks; the AI feature additionally requires the Gemini API key.

# 08 | Limitations and Future Scope

## Current limitations

- The current task workflow focuses on task descriptions and completion status; due dates and reminders are not part of the documented feature set.
- AI-generated steps are suggestions and may need user review before being used.
- The application depends on external services, including database hosting, deployment hosting, and the AI API.
- Local setup requires the user to configure environment variables correctly.

## Possible future improvements

- Add task deadlines, priority levels, and reminder notifications.
- Add search, tags, categories, or project-based grouping.
- Provide a way to reorder tasks and track progress over time.
- Improve error messages and provide clearer feedback when an external service is unavailable.
- Add more automated tests for authentication, task routes, and database operations.
- Improve accessibility with keyboard support, focus states, and screen-reader labels.

## Security considerations

The project uses account authentication and environment-based configuration. For a production-ready system, authentication, session settings, input validation, API error handling, and access controls should be reviewed regularly. API keys and database credentials should remain private and must never be embedded in frontend JavaScript or committed to the public repository.

# 09 | Conclusion and References

## Conclusion

The Todo List Application brings together a responsive frontend, a Node.js and Express backend, MongoDB Atlas storage, account access, and AI-assisted task planning. Building it provided practical experience with frontend-to-backend communication, CRUD operations, persistent data, environment variables, deployment, and connecting an external AI service. The calm visual design supports the project's goal: making everyday task planning feel clearer and more manageable.

## Project links

**Live application:** https://todo-app-mfnx.onrender.com

**GitHub repository:** https://github.com/Shifa-Khan-15/Todo-App

## References


- Node.js documentation — https://nodejs.org/docs/
- Express.js documentation — https://expressjs.com/
- MongoDB documentation — https://www.mongodb.com/docs/
- Mongoose documentation — https://mongoosejs.com/docs/
- Google AI for Developers — https://ai.google.dev/
- Render documentation — https://render.com/docs
- GitHub documentation — https://docs.github.com/


> **Make today a little lighter.**